Slotit: Sports Venue Booking & Tournament Management System

A PROJECT REPORT

submitted by

Ebin Biju Abraham, B24ELA21
Afroz Shanavas Palott, B24ELA09
Niveditha C M, B24ELA52
Tharun Bhaskaran, B24ELA66
(Group 9)

as part of the project based course

24ERJ502 : Database Management Systems

in partial fulfillment of the requirements for the award of the Degree
of
Bachelor of Technology
In
Electrical and Computer Engineering



DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING
T.K.M. COLLEGE OF ENGINEERING, KOLLAM
OCTOBER, 2026

ABSTRACT

This project is about **Slotit**, a Sports Venue Booking and Tournament Management System. Many sports turfs and courts still take bookings through phone calls and WhatsApp messages. This causes problems like two people booking the same slot (double booking), no clear pricing for peak hours, and tournament fixtures being drawn by hand on paper. Slotit tries to solve these problems using proper database concepts.

The database is built in MySQL 8.0 with 12 tables in Third Normal Form (3NF). To stop double booking, we use row-level locking with `SELECT ... FOR UPDATE` inside a JDBC transaction, so every booking follows the ACID properties. Two triggers are used: one increases the booking streak of a player, and the other sends a notification to waitlisted users when a booked slot becomes free. Nearby venues are found by writing the Haversine formula directly in an SQL query. A MongoDB script defines three collections (player stats, match commentary and turf photos), but the Java code currently uses mock data for these.

The backend has 9 Java Servlets and 7 DAO classes running on Apache Tomcat 9. The frontend is a mobile-first Progressive Web App with 14 screens. The other features are wallet payment with refunds, HotSlot peak and off-peak pricing, weather advisory, squad finder, tournament registration with automatic knockout fixtures, an owner dashboard, and an offline fallback. The two-session concurrency test shows that the row lock allows only one booking for a slot.

Contents

Chapter 1: Introduction 5
1.1 Background 5
1.2 Problem Statement 5
1.3 Objectives of the Project 5
1.4 Scope of the Project 6
Chapter 2: Relevance of the Project 7
2.1 Introduction 7
2.2 Relevance 7
2.3 Conclusion 7
Chapter 3: Methodology 8
3.1 Introduction 8
3.2 System Architecture 8
3.3 Component Description 9
3.4 ER Diagram 11
3.5 Relational Schema and 3NF Normalization 15
3.6 Stage-by-Stage Design Description 16
3.7 Application Screens 20
3.8 Implementation and Deployment 21
3.9 Conclusion 22
Chapter 4: Results and Discussion 23
4.1 Introduction 23
4.2 Concurrency Test and Double-Booking Prevention 23
4.3 Wallet, Cancellation and Refund 23
4.4 Dynamic Pricing and Waitlist Notification 23
4.5 Geospatial Search 24
4.6 Tournament and Squad Features 24
4.7 Offline Behaviour of the App 24
4.8 Data Loaded in the Database 24
4.9 Challenges and Solutions 25
4.10 Limitations and Future Scope 25
4.11 Conclusion 26
References 27
Appendix 28

List of Figures

Figure 3.1: Three-tier architecture of Slotit 8
Figure 3.2: ER diagram of Slotit (primary keys underlined, foreign keys in italics) 12
Figure 3.3: Two transactions trying to book the same slot 15

List of Tables

Table 3.1: The three tiers of Slotit 9
Table 3.2: Tables of the Slotit database 13
Table 3.3: Triggers and transactional routines 16
Table 3.4: Screens of the Slotit application 19
Table 4.1: Behaviour with and without row locking (two-session test) 21
Table 4.2: Haversine radius search behaviour 22
Table 4.3: Sample data in slotit_db 23
Table 4.4: Problems faced and how they were solved 23
Table A.1: REST API endpoints 26
Table A.2: Software tools and versions 27
Table A.3: The ten analytical queries 27
Table A.4: Files of the project 27
Table A.5: Specification of the users table 28
Table A.6: Specification of the venues and courts tables 28
Table A.7: Specification of the time_slots and bookings tables 29
Table A.8: Specification of the remaining seven tables 29

Chapter 1
Introduction

1.1 Background

In the last few years, many sports turfs and courts have opened in India. There are floodlit football turfs, synthetic badminton courts, box-cricket arenas, tennis courts and basketball courts. Most of them rent out the ground by the hour, and the price is higher in the evening and on weekends because more people want to play at that time.

Even though so many people use these venues, the way they are managed is still very old. Most owners take bookings over phone calls, write them in a register, or talk to customers on WhatsApp. This works when there are very few customers, but it creates a lot of confusion when many people try to book at the same time.

In our DBMS classes we mostly learn about tables, keys and queries using small examples with only one user. A real system is different. Many users send requests at the same moment, the network can drop, and money (wallet balance) is involved. We wanted to see how the database concepts we studied actually work in such a situation. So we built **Slotit**, a sports venue booking and tournament management system, with the database as the most important part of it.

1.2 Problem Statement

A venue that does not use a proper database system faces many problems. The main ones we found are:

- **Double booking:** When two people try to book the same slot at almost the same time, a simple program reads the slot as 'available' for both of them. Both bookings get confirmed and two teams reach the turf together.
- **Hard to find venues:** Players have to call many owners to find a free court. There is no easy way to see the nearest venue first.
- **Unbalanced use of courts:** Evening slots are always full, but morning and afternoon slots stay empty because the price is the same for all hours.
- **Cancelled slots are missed:** When someone cancels a prime-time slot, the other players who wanted it do not come to know.
- **Manual tournaments:** Organisers collect team names on paper, draw the fixtures by hand, and sometimes allow duplicate teams or more teams than the bracket can hold.
- **Incomplete squads:** A player who has only 4 friends for a 7-a-side game cannot find the remaining players easily.
- **Security and network problems:** Passwords may be stored carelessly, and a normal web page just hangs when the Wi-Fi is weak.

1.3 Objectives of the Project

The main objectives of the Slotit project are:

- To design a 12-table relational database in MySQL 8.0 that follows Third Normal Form (3NF) and keeps the data correct using primary keys, foreign keys and constraints.
- To stop double booking by using pessimistic row-level locking (`SELECT ... FOR UPDATE`) inside JDBC transactions, so that bookings follow the ACID properties.
- To keep the wallet deduction, slot update and booking record together in one transaction, so that either everything happens or nothing happens.
- To find nearby venues by writing the Haversine distance formula directly in an SQL query.
- To automate some rules inside the database using two triggers (booking streak and waitlist notification).
- To add peak and off-peak pricing (HotSlot), tournament registration with automatic knockout fixtures, and a squad finder.
- To use MongoDB along with MySQL (polyglot persistence) for data that does not fit well in fixed tables, like player stats, match commentary and turf photos.
- To build a mobile-friendly Progressive Web App (PWA) that can be installed on a phone and still works when the network is slow or disconnected.

1.4 Scope of the Project

Slotit covers the full path from a player opening the app to the booking being stored in the database. The project includes the following parts:

- **Player side:** Register and login, choose a city or use GPS, search venues by name or sport, see courts and the slots for the next 7 days, pay from a wallet, view and cancel bookings, join or post squad requests, and register teams for tournaments.
- **Venue owner side:** A dashboard that shows total bookings, revenue, active courts, peak-hour use, a 7-day revenue chart and recent bookings.
- **Admin side:** An overview page with platform details.
- **Tournament module:** Team registration with capacity and duplicate-name checks, and automatic round-1 knockout fixtures.
- **Cities covered:** Kollam, Kochi, Trivandrum and Calicut (17 venues in total).

The project runs as a web application on Apache Tomcat in a local network. It is not yet connected to a real payment gateway, and the MongoDB part uses sample data in the Java code. These points are discussed again in Chapter 4.

Chapter 2
Relevance of the Project

2.1 Introduction

Almost every modern application depends on a database. The screens can look very nice, but if the database design is weak, the data becomes wrong, the money does not match and bookings get duplicated. That is why the Database Management Systems course is important, and Slotit tries to show these ideas in a working system instead of only on paper.

2.2 Relevance

This project is directly related to the course 24ERJ502 Database Management Systems for the following reasons:

- **ER modelling and relational schema:** We first identified the entities (users, venues, courts, slots, bookings, tournaments and so on) and their relationships, and then converted the ER diagram into 12 tables with primary and foreign keys.
- **Normalization:** The tables are in 1NF, 2NF and 3NF. For example, the hourly rate is stored once in the courts table and not repeated in every time slot, and the squad_members table removes the many-to-many problem between squads and players.
- **Transactions and ACID:** Booking and cancelling use JDBC transactions with commit and rollback, which shows atomicity, consistency, isolation and durability.
- **Concurrency control:** Double booking is a real example of the lost update problem. We solved it using row-level exclusive locks, which is the idea of two-phase locking taught in the course.
- **Triggers:** Two MySQL triggers run automatically after an insert or update, without any extra code in the application.
- **SQL queries:** The project uses joins, GROUP BY, HAVING, subqueries, CASE expressions and a view. Ten such queries are kept in useful_queries.sql.
- **Indexing:** Five indexes (for example on email, date and status, and on latitude and longitude) make searching faster.
- **Spatial calculation in SQL:** The Haversine formula is used inside a query to sort venues by distance.
- **NoSQL and polyglot persistence:** MongoDB is used beside MySQL to show where a document database is more suitable than a relational one.

2.3 Conclusion

Slotit connects the theory of the DBMS course with a working application. Concepts like normalization, locking, triggers and indexing are not just learnt for the exam; they are used to solve real problems like double booking and refunds. This makes the project useful both for learning and as a base for a real venue booking system.

Chapter 3
Methodology

3.1 Introduction

This chapter explains how Slotit was designed and built. It starts with the overall architecture, then the tools used, the ER diagram, the tables and normalization, and then each important feature one by one (booking with locking, wallet, pricing, triggers, search, tournaments, squads, security and the offline app). At the end it explains how the project was deployed and installed on a phone.

3.2 System Architecture

Slotit follows a three-tier design. The first tier is the app that the user sees on the phone or laptop. The second tier is the Java application running on Apache Tomcat, which receives requests and talks to the database. The third tier is the data storage: MySQL for all important transactional data and MongoDB for flexible data like stats and photos. Figure 3.1 shows the three tiers.

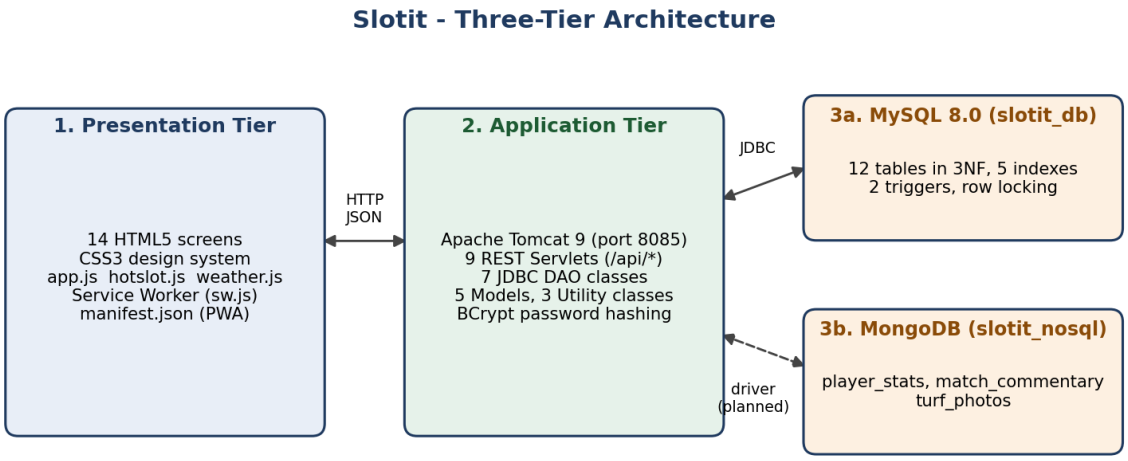


Figure 3.1: Three-tier architecture of Slotit

Tier	Technologies used	What it does in Slotit
1. Presentation (client)	14 HTML5 pages, CSS3, vanilla JavaScript (app.js, hotslot.js, weather.js), manifest.json, Service Worker (sw.js)	Shows the screens, asks for GPS location, calculates the peak/off-peak price badges, shows the weather advisory, caches data and falls back to saved data when the network fails.
2. Application (middle)	Java 11, Servlet 4.0, Apache Tomcat 9.0.122 on port 8085, 9 servlets, 7 DAO classes, 5 model classes, 3 utility classes	Gives JSON REST APIs under /Slotit/api/*, checks the login using BCrypt, adds CORS headers and runs the JDBC transactions.

Tier	Technologies used	What it does in Slotit
3. Data	MySQL 8.0 (slotit_db) and MongoDB (slotit_nosql)	MySQL stores 12 normalized tables, indexes and 2 triggers and handles row locking. MongoDB stores player stats, match commentary and turf photos.

Table 3.1: The three tiers of Slotit

How one action travels through the system: The steps below show what happens when a player books a slot.

1. The player opens Slotit in the phone browser and taps **Confirm & Pay** on the checkout screen.
2. The JavaScript function `apiCall()` sends a `fetch()` request with a JSON body to Tomcat.
3. Tomcat passes the request to `BookingServlet`, which reads the values using `JsonUtil` and calls `BookingDAO`.
4. `BookingDAO` takes a JDBC connection, turns off auto-commit, locks the slot row with `SELECT ... FOR UPDATE` and runs the updates.
5. Inside MySQL, the trigger `trg_update_streak` fires automatically after the booking row is inserted.
6. The servlet sends back a JSON reply with status 200, 201 or 409. The app clears its cache, updates the wallet balance and shows the result.

3.3 Component Description

This section describes the main tools and components used in the project, along with their important details.

3.3.1 MySQL 8.0 (Relational Database)

MySQL stores everything that needs to be correct and consistent: users and wallets, venues, courts, slots, bookings, tournaments and squads. The InnoDB engine is used because it supports transactions, foreign keys and row-level locking.

Parameter	Value
Database name	slotit_db (UTF8MB4 character set)
Storage engine	InnoDB (transactions, foreign keys, row locks)
Tables	12, all in Third Normal Form
Indexes	5 (email, venue location, slot date + status, booking user, tournament venue)
Triggers	2 (trg_update_streak, trg_slot_drop_notify)
Port	3306 (MySQL80 Windows service)
Script files	schema.sql, triggers.sql, useful_queries.sql

3.3.2 MongoDB (Document Database)

Some data like sport-specific player stats, commentary lines and photo details do not have the same shape for every record. A football player has goals and assists but a cricketer has runs and wickets. Putting all of this into SQL tables would leave many empty columns, so we defined these in MongoDB.

Collection	What it stores
player_stats	Per-user and per-sport numbers like matches played, goals, assists, win rate and an achievements array
match_commentary	Time-stamped commentary lines for a match
turf_photos	Photo details, captions and ratings for each venue

The collections and sample documents are created by `mongodb_setup.js` for the database `slotit_nosql`. The `UserStatsDAO` class currently returns mock data; the MongoDB driver calls are written but kept commented, so the Java code is not yet connected to MongoDB.

3.3.3 Java Servlets, DAO Classes and Utilities

The Java code is organised in packages. The servlets receive HTTP requests, the DAO (Data Access Object) classes contain all the SQL, the models are simple Java objects, and the utility classes help with the database connection, passwords and JSON.

Class	Package	Main work
UserDAO	com.slotit.dao	Register user, find by email, check login, update wallet
VenueDAO	com.slotit.dao	Find all venues, by sport, by id and nearby venues using Haversine
SlotDAO	com.slotit.dao	Get courts of a venue and the slots of a date with the calculated price
BookingDAO	com.slotit.dao	Book a slot, cancel and refund, booking history, owner dashboard numbers
TournamentDAO	com.slotit.dao	List tournaments and teams, register a team, generate knockout fixtures, update match score
SquadDAO	com.slotit.dao	Open squad requests, create a request, join a squad
UserStatsDAO	com.slotit.dao	Player stats, commentary and turf photos (MongoDB side, mock data now)
User, Venue, Court, TimeSlot, Booking	com.slotit.models	Five model classes with toJson() methods
DBConnection, PasswordUtil, JsonUtil	com.slotit.util	Connection factory using environment variables, BCrypt and SHA-256 hashing, a small JSON parser and escaper

3.3.4 Apache Tomcat and Maven

Apache Tomcat 9.0.122 is the web container that runs the servlets. We set its HTTP connector to port 8085 so that it does not clash with other programs on port 8080. Apache Maven 3.9.6 compiles the 24 Java files and packs them with the MySQL driver (`mysql-connector-j` 8.0.33) into one file called `Slotit.war`.

3.3.5 Progressive Web App (Frontend)

The frontend uses plain HTML5, CSS3 and JavaScript, without any big framework. It has three important JavaScript files and the PWA files:

- **app.js:** The main file. It has the `apiCall()` function, login check for protected pages, toast messages, the 30-second GET cache, the 900 ms timeout, the 2-minute offline circuit breaker and a built-in catalogue of 17 venues and 34 courts.
- **hotslot.js:** Calculates the final price of a slot, shows the badges (Booked, Peak, Off-Peak Deal) and changes 24-hour times like 18:00:00 into 6:00 PM.
- **weather.js:** Gets the forecast from the OpenWeatherMap API for the venue location and the selected date, and shows a weather message.
- **manifest.json and sw.js:** Make the site installable on a phone and let it work with cached files.

3.4 ER Diagram

From the problem we identified the entities and how they are related. A user who is an owner owns many venues. A venue has many courts, and each court has many one-hour time slots. A time slot can be reserved by one booking at a time. A tournament belongs to a venue and has many teams and many matches. Players can also post squad requests for a slot, and other players join them. The ER diagram is shown in Figure 3.2.

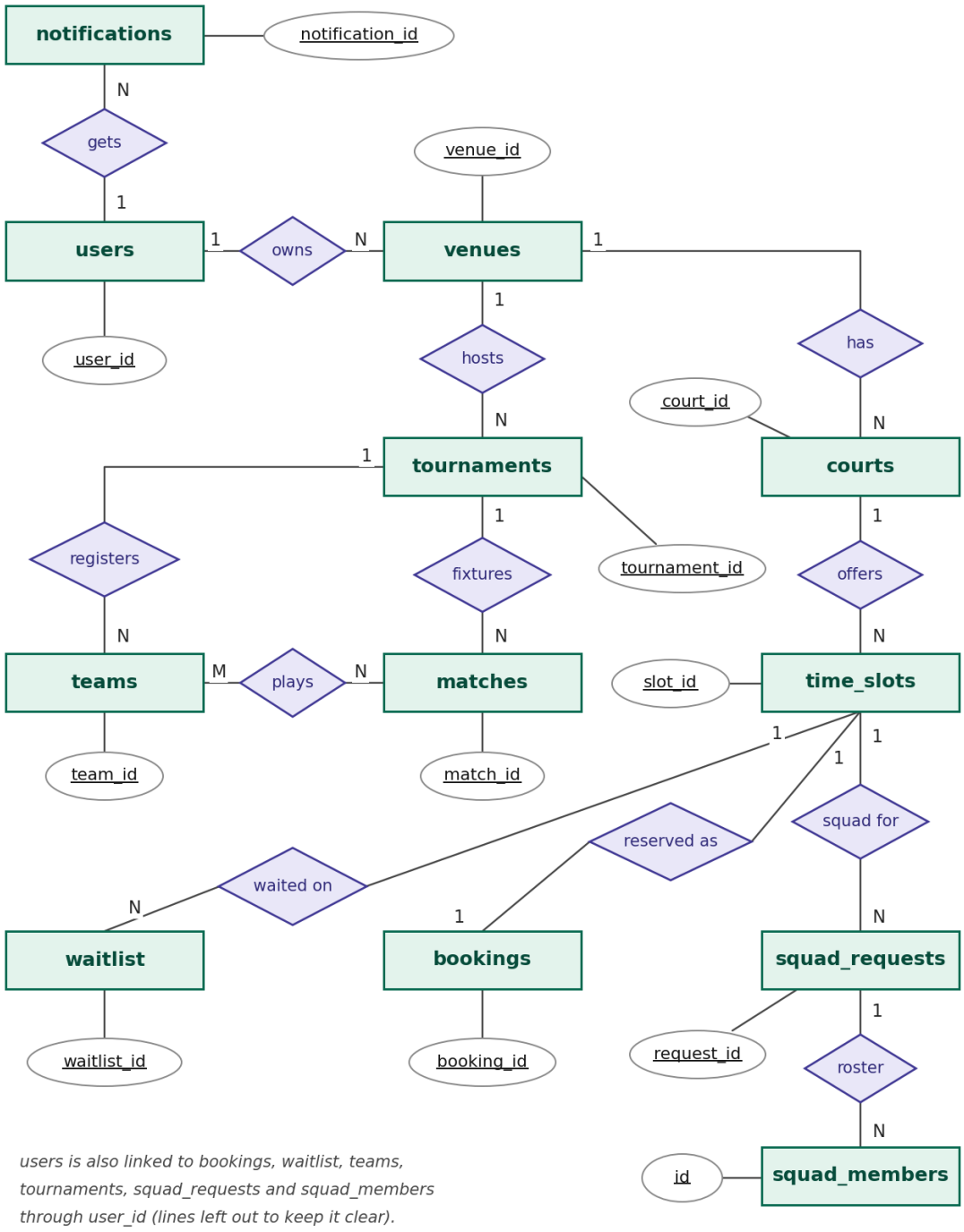
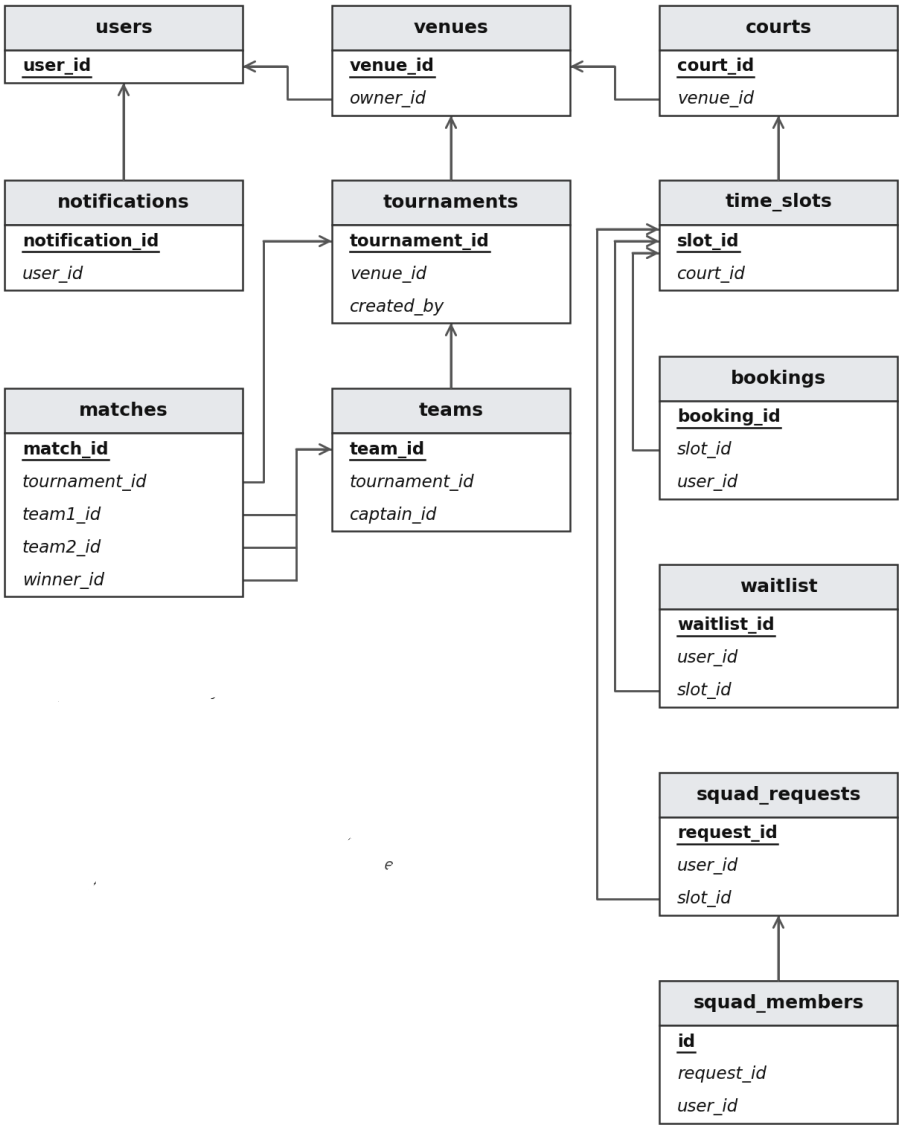


Figure 3.2: ER diagram of Slotit (primary keys underlined, foreign keys in italics)

Relational Schema



In the diagram, each arrow starts from the table that holds the foreign key and points to the table it refers to.

The main relationships are:

- User (1:N) Venue: one owner can own many venues.
- Venue (1:N) Court, and Court (1:N) TimeSlot.
- TimeSlot (1:N) Booking: a slot has only one active (completed) booking at a time, but cancelled and refunded bookings stay in the table, so the same slot_id can appear in more than one booking row.
- TimeSlot (1:N) Waitlist and TimeSlot (1:N) SquadRequest.
- SquadRequest (1:N) SquadMember: this table resolves the many-to-many relation between players and squads.

- Tournament (1:N) Team and Tournament (1:N) Match.
- Team (1:N) Match: matches stores team1_id, team2_id and winner_id as foreign keys to teams.

3.5 Relational Schema and 3NF Normalization

The ER diagram was converted into 12 tables. Table 3.2 lists each table with its primary key, foreign keys and purpose. Column-level details (data types and constraints) are given in Appendix F.

Table	Primary key	Foreign keys / constraints	Purpose
users	user_id	UNIQUE(email); role: player, owner, admin; badge: none, silver, gold, platinum	Accounts, password hash, wallet balance, booking streak, loyalty badge
venues	venue_id	owner_id -> users	Sports complexes with latitude, longitude, sport type, rating, indoor or outdoor
courts	court_id	venue_id -> venues	Playing surfaces with hourly_rate and peak_multiplier
time_slots	slot_id	court_id -> courts; UNIQUE(court_id, slot_date, start_time)	One-hour slots with status (available, booked, maintenance) and is_peak flag
bookings	booking_id	user_id -> users; slot_id -> time_slots	Reservation record with amount_paid, payment_status and time
tournaments	tournament_id	venue_id -> venues; created_by -> users	Events with sport, dates, format (knockout, league), max_teams and status
teams	team_id	tournament_id -> tournaments; captain_id -> users	Teams registered in a tournament
matches	match_id	tournament_id, team1_id, team2_id, court_id, winner_id	Fixtures, scheduled time, scores and winner
waitlist	waitlist_id	user_id -> users; slot_id -> time_slots	Users waiting for a booked slot
squad_requests	request_id	user_id -> users; slot_id -> time_slots	Open teammate posts with players_needed, current_count and status
squad_members	id	UNIQUE(request_id, user_id)	Players who joined a squad (stops duplicate joins)
notifications	notification_id	user_id -> users (ON DELETE CASCADE)	Alerts created by the trigger

Table 3.2: Tables of the Slotit database

How the tables follow the normal forms:

- **1NF:** Every column holds a single value. Things that can repeat, like the players of a squad or the slots of a court, are kept in separate tables.
- **2NF:** In squad_members, the details of the squad and the player are not stored again; only the two keys are stored, so no column depends on a part of a key.

- **3NF:** The descriptive data is not repeated. The hourly rate and peak multiplier are stored only in courts (not in time_slots), and the venue address and GPS location are stored only in venues (not in courts or tournaments).
- **Deliberate exceptions:** A few columns are kept even though they can be calculated from other data, to make reading faster: `users.total_bookings`, `users.streak_count` and `users.badge` (the badge depends on total_bookings), `time_slots.is_peak` (depends on start_time), `squad_requests.current_count` and `teams.player_count`. Strictly, these are small controlled denormalizations, and they are always updated inside the same transaction as the data they come from.

Indexes used (schema.sql):

- `idx_user_email` on users(email) for fast login lookup.
- `idx_venue_location` on venues(latitude, longitude) for the distance search.
- `idx_slot_date_status` on time_slots(slot_date, status) to load the daily slot grid quickly.
- `idx_booking_user` on bookings(user_id) for booking history.
- `idx_tournament_venue` on tournaments(venue_id).

3.6 Stage-by-Stage Design Description

3.6.1 Booking with Row-Level Locking

This is the most important part of the project. If the code only reads the slot and then updates it, two users can both read 'available' at the same time. To stop this, `BookingDAO.bookSlot()` turns off auto-commit and locks the slot row using `SELECT ... FOR UPDATE`. InnoDB gives the exclusive lock to the first transaction, and the second one has to wait. There is also a second safety at the schema level: `UNIQUE (court_id, slot_date, start_time)` so a duplicate slot row can never be created.

```
conn.setAutoCommit(false);
try {
  // 1. Lock the slot row (and read its price details)
  String lockSql = "SELECT ts.status, ts.is_peak, c.hourly_rate, c.peak_multiplier "
      + "FROM time_slots ts JOIN courts c ON ts.court_id = c.court_id "
      + "WHERE ts.slot_id = ? FOR UPDATE";
  ResultSet rs = lockStmt.executeQuery();
  if (!rs.next() || !"available".equalsIgnoreCase(rs.getString("status"))) {
      conn.rollback();
      return new BookingResult(false, "Slot is no longer available");
  }
  // 2. Lock the user row, check wallet, deduct the amount
  // 3. Update slot status to 'booked'
  // 4. Insert the booking record (trigger updates the streak)
  conn.commit();            // lock is released here
} catch (Exception e) { conn.rollback(); throw e; }
```

Listing 3.1: Row-level locking in BookingDAO (simplified)

The steps of two users booking slot 101 at the same time are shown in Figure 3.3.

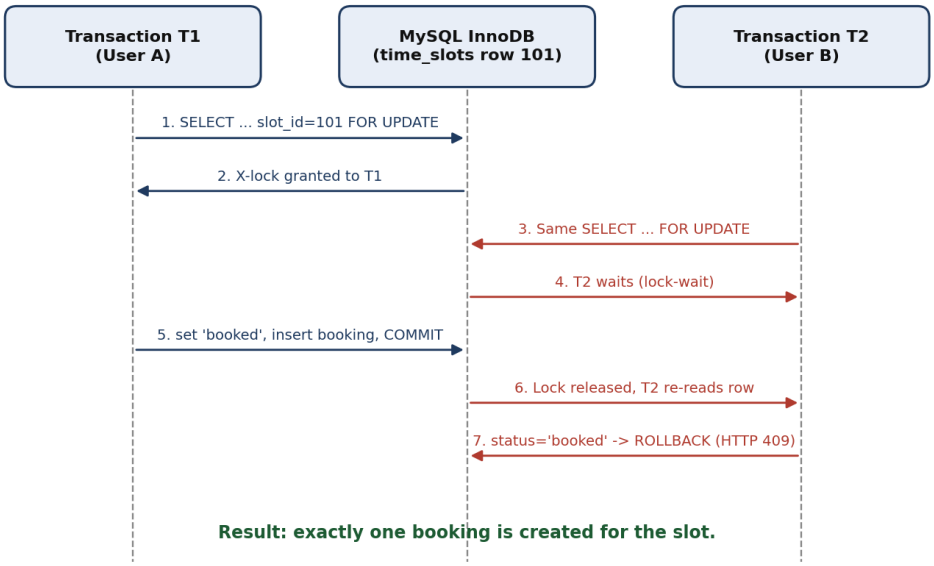


Figure 3.3: Two transactions trying to book the same slot

To avoid deadlocks, we always lock in the same order: first the time_slots row and then the users row.

3.6.2 Wallet, Cancellation and Refund (Atomicity)

A booking changes three tables: the slot status in time_slots, the wallet and total bookings in users, and a new row in bookings. Cancelling does the reverse: the booking is marked 'refunded', the slot becomes available again and the money goes back to the wallet. If the server stops in the middle, the user could lose money without getting a slot. So both `bookSlot()` and `cancelBooking()` run as a single transaction. The slot row and the user row are locked first. If the wallet balance is less than the price, or any SQL step fails, `conn.rollback()` brings the database back to its old state. Past bookings and already refunded bookings cannot be cancelled.

3.6.3 HotSlot Peak and Off-Peak Pricing

Evening slots are always in demand while the morning ones stay empty. Each court has a base `hourly_rate` and a `peak_multiplier` (for example 1.50 for football and cricket turfs, 1.25 for indoor badminton and 1.30 for tennis). Each slot has an `is_peak` flag, which is TRUE for the evening hours 17:00 to 21:00. The price is found using a CASE expression in SQL:

```
CASE WHEN ts.is_peak THEN ROUND(c.hourly_rate * c.peak_multiplier, 2)
     ELSE c.hourly_rate END AS final_price
```

Listing 3.2: Final price calculation in SQL

The same calculation is done in `hotslot.js` so the screen shows a **Peak** badge on evening slots and an **Off-Peak Deal** badge on daytime slots, to encourage people to book in the free

hours. For example, a court with a rate of Rs 800 and multiplier 1.5 costs Rs 1,200 at peak time and Rs 800 at other times.

3.6.4 Triggers and Transactional Routines

Two triggers keep some rules inside the database so that they work no matter which program changes the data. The other multi-step rules are kept in JDBC transactions in the DAO classes.

Object	Type	What it does
trg_update_streak	Trigger (AFTER INSERT on bookings)	Increases streak_count of the player by 1 after each booking.
trg_slot_drop_notify	Trigger (AFTER UPDATE on time_slots)	When a slot changes from 'booked' to 'available', it inserts a notification row for every user in the waitlist of that slot. No cron job or polling is needed.
BookingDAO.bookSlot	JDBC transaction	Locks slot and user rows, checks the balance, deducts the amount, updates total_bookings and badge (silver, gold, platinum at 5, 10 and 25 bookings), marks the slot booked and inserts the booking.
BookingDAO.cancelBooking	JDBC transaction	Marks the booking refunded, frees the slot (which fires the notify trigger) and refunds the wallet.
TournamentDAO.registerTeam	JDBC transaction	Locks the tournament row, checks capacity and duplicate team name, inserts the team.
TournamentDAO.generateKnockoutFixtures	JDBC transaction	Shuffles the teams, pairs them and batch-inserts round-1 matches.
SquadDAO.joinSquad	JDBC transaction	Locks the squad row, blocks duplicate joins, increases current_count and marks the squad full when needed.

Table 3.3: Triggers and transactional routines

The definitions of the two triggers (from triggers.sql) are given below.

```
CREATE TRIGGER trg_update_streak AFTER INSERT ON bookings
FOR EACH ROW
  UPDATE users SET streak_count = streak_count + 1
  WHERE user_id = NEW.user_id;

DELIMITER //
CREATE TRIGGER trg_slot_drop_notify AFTER UPDATE ON time_slots
FOR EACH ROW
BEGIN
  IF OLD.status = 'booked' AND NEW.status = 'available' THEN
    INSERT INTO notifications (user_id, message)
    SELECT user_id, CONCAT('Slot ID ', NEW.slot_id, ' on ', NEW.slot_date,
           ' at ', NEW.start_time, ' is now available!')
    FROM waitlist WHERE slot_id = NEW.slot_id;
  END IF;
END//
DELIMITER ;
```

Listing 3.3: Definitions of the two triggers

Note that `streak_count` goes up by 1 on every new booking, so it is a running count of bookings made, not a count of consecutive days.

3.6.5 Nearby Venue Search using Haversine Distance

To show venues sorted by real distance, a great-circle distance formula is written inside the SQL query in `VenueDAO.findNearby(lat, lng, radiusKm)`. This avoids paid map services. Strictly speaking, the query is written in the spherical law of cosines form, which gives the same great-circle distance as the Haversine formula and is easier to write in SQL; we call it the Haversine distance in this report. The `LEAST` and `GREATEST` functions keep the value of `acos()` between -1 and 1, because floating point rounding can sometimes push it slightly outside and cause an error. The index `idx_venue_location` supports the search.

```
SELECT v.*, (6371 * ACOS(LEAST(1.0, GREATEST(-1.0,
        COS(RADIANS(?)) * COS(RADIANS(v.latitude)) *
        COS(RADIANS(v.longitude) - RADIANS(?)) +
        SIN(RADIANS(?)) * SIN(RADIANS(v.latitude)))))) AS distance_km
FROM venues v
HAVING distance_km <= ?
ORDER BY distance_km ASC;
```

Listing 3.4: Haversine distance query (simplified from VenueDAO)

3.6.6 Tournament Registration and Knockout Fixtures

Registration locks the tournament row with `FOR UPDATE`, checks that the tournament is not completed or cancelled, compares the current team count with `max_teams`, and checks the team name in a case-insensitive way (`LOWER(name) = LOWER(?)`) before inserting. Because of the lock, two captains registering together cannot go beyond the maximum. To generate the bracket, `generateKnockoutFixtures()` takes all team ids, shuffles them using `Collections.shuffle()`, pairs the neighbouring teams, gives each pair a court of the tournament venue and inserts all matches together using a JDBC batch (`addBatch()` and `executeBatch()`). It refuses to run if fixtures already exist or if fewer than two teams are registered.

3.6.7 Squad Finder

A player who needs more teammates posts a squad request with the sport, the slot, a message and the number of players needed. Other players see it in the Squad Finder screen and tap **Join Squad**. `SquadDAO.joinSquad()` locks the request row, rejects a player who already joined (also enforced by the UNIQUE key on request_id and user_id), inserts the member, increases `current_count`, and changes the status to 'full' when the needed number is reached. This way the squad cannot be over-filled.

3.6.8 Security

- **Password hashing:** `PasswordUtil.java` has its own BCrypt ($2a$10$) with a random salt, so we did not need any external library. It can also check the old 64-character SHA-256 hashes used in the seed data.

- **Role safety:** `RegisterServlet` accepts only 'player' or 'owner', so nobody can make themselves an admin during sign-up.
- **Database credentials:** The database URL, user and password are not written in the Java code. `DBConnection.java` reads them from the environment variables `SLOTIT_DB_URL`, `SLOTIT_DB_USER` and `SLOTIT_DB_PASSWORD`, which are set in Tomcat's `setenv.bat`.
- **SQL injection:** All queries use `PreparedStatement` with parameters.

3.6.9 Owner Dashboard, Profile and Loyalty

The owner dashboard (`GET /api/owner/dashboard`) uses joins and aggregate functions to return four numbers: total bookings, total revenue in rupees, active courts and peak hour utilization. It also returns a 7-day daily revenue chart and the recent bookings with customer names. The profile page shows the wallet balance, total bookings, the booking streak (updated by the trigger) and the loyalty tier badge (silver, gold or platinum).

3.6.10 Offline Support and Weather Advisory

Slotit tries not to hang when the network is weak. In `app.js` there are three layers of protection: (1) GET responses are cached in sessionStorage for 30 seconds and cleared whenever a POST or DELETE changes the data; (2) every request uses an `AbortController` with a 900 ms timeout; and (3) if requests fail, a circuit breaker (`slotit_offline_until`) stops calling the server for 2 minutes and the app shows the built-in catalogue of 17 venues with an offline message. The service worker `sw.js` pre-caches the application pages in the cache `slotit-app-v5`. On the venue page, `weather.js` shows a message such as indoor venue is not affected, rain expected, or good weather for outdoor play.

3.6.11 Polyglot Persistence

MySQL holds the data that must be exact (wallets, slots, bookings), and MongoDB is meant for the flexible data (player stats, commentary, photos). They are linked using the ids `user_id`, `match_id` and `venue_id`. As explained in Section 3.3.2, the Java side currently uses sample data for this part.

3.7 Application Screens

The app has 14 pages. Table 3.4 gives the main features of each page.

Page	Used by	Main features
splash.html	Everyone	Start screen with logo and tagline 'Book Turfs. Build Squads. Win Tournaments.'; sends a logged-in user directly to home or dashboard.
login.html	All roles	Email and password login, role-based redirect, quick-fill buttons for demo player and demo owner.
register.html	New users	Sign-up with name, email, 10-digit phone, password confirmation and a Player / Venue Owner toggle.

Page	Used by	Main features
location.html	Everyone	Use live GPS with distance sorting, or choose one of 4 cities: Kollam (5 venues), Kochi (4), Trivandrum (4) and Calicut (4).
home.html	Players	Search by venue or area, sport filter chips (Football, Badminton, Cricket, Basketball, Tennis), venue cards with rating, distance and starting rate, links to Tournaments and Squad Finder.
venue-detail.html	Players	Court selector, 7-day date strip, weather advisory and slot grid with Available, Peak, Off-Peak Deal and Booked states.
checkout.html	Players	Venue, court, date and time summary, base rate plus peak surcharge, wallet balance, Confirm & Pay button.
my-bookings.html	Players	Upcoming and Past/Cancelled tabs, payment status badge, Cancel & Refund button.
tournaments.html	Players, captains	Upcoming, ongoing and completed tournaments with capacity bar (for example 5/8 teams), team registration form and team list.
squad-finder.html	Players	Open squad posts with sport, venue, time and spots left; Join button and a form to post a new squad request.
profile.html	Players, owners	Avatar, role, loyalty badge, booking streak, total bookings, wallet balance and player stats.
owner-dashboard.html	Venue owners	4 KPI cards, 7-day revenue bar chart and recent bookings table.
admin.html	Admin	Platform overview: metrics, venue approvals, user roles and database status.
index.jsp	Everyone	Entry point that redirects /Slotit/ to splash.html.

Table 3.4: Screens of the Slotit application

3.8 Implementation and Deployment

The project was run on a Windows laptop and used from a phone on the same Wi-Fi network. The steps we followed are given below.

Step 1: Set up the MySQL database

```
net start MySQL80
mysql -u root -p < "Slotit/database/schema.sql"
mysql -u root -p slotit_db < "Slotit/database/triggers.sql"
mysql -u root -p -e "USE slotit_db; SHOW TABLES; SHOW TRIGGERS;"
```

Step 2: Keep database credentials outside the code

A file `bin\setenv.bat` was created in the Tomcat folder with the three environment variables, so Tomcat sets them every time it starts.

```
@echo off
set
"SLOTIT_DB_URL=jdbc:mysql://localhost:3306/slotit_db?useSSL=false&allowPublicKeyRetr
ieval=true&serverTimezone=Asia/Kolkata"
set "SLOTIT_DB_USER=root"
set "SLOTIT_DB_PASSWORD=<your_mysql_password>"
```

Step 3: Build the WAR file with Maven

```
cd Slotit
mvn clean package
REM output: target\Slotit.war
```

Step 4: Deploy on Tomcat (port 8085)

The connector port in `conf\server.xml` was changed to 8085 and the WAR file was copied to the `webapps` folder. Tomcat unpacks it by itself and the application is available at `/Slotit`.

Step 5: One-click launcher

To start everything easily before a demo, we made `Start_Slotit_Server.bat`. When run as administrator, it starts the MySQL80 service, opens port 8085 in Windows Firewall, sets JAVA_HOME and the database variables, restarts Tomcat, and prints the laptop and phone URLs.

Step 6: Use it on a phone as an app

- Connect the laptop and the phone to the same Wi-Fi (or the phone's hotspot).
- Open `http://<laptop-IP>:8085/Slotit/splash.html` in Chrome or Safari on the phone.
- Choose **Add to Home screen** from the browser menu to install it as an app.
- Chrome normally allows full PWA installation only on HTTPS or localhost. For the plain HTTP address used in the lab, we turned on the flag `chrome://flags/#unsafely-treat-insecure-origin-as-secure` for the laptop address. We also created 192x192 and 512x512 PNG icons (icon-192.png, icon-512.png) in manifest.json and registered sw.js.

Two demo accounts (one player and one venue owner) were added in the seed data so the player and owner screens can be shown quickly.

Showing database changes live: While using the app, we kept a MySQL window open and ran queries like `SELECT ... FROM users ORDER BY user_id DESC LIMIT 5;` and `SELECT * FROM notifications ORDER BY notification_id DESC LIMIT 5;`. After every booking, cancellation, or squad join, the changes were visible in the tables.

3.9 Conclusion

The methodology builds Slotit step by step, starting from the ER design and the 3NF tables, then adding locking, triggers, pricing, search, tournaments, squads, security and the offline app. Each important rule is kept in the database or in a transaction so that the data stays correct even when many users act at once.

Chapter 4
Results and Discussion

4.1 Introduction

This chapter explains how the main features of Slotit were tested and what we observed. The tests were done by using the app on a laptop and on a phone, and by running SQL statements in two MySQL sessions at the same time. We did not record load or timing benchmarks, so no speed numbers are claimed here.

4.2 Concurrency Test and Double-Booking Prevention

The double-booking situation was shown using two database sessions (script `concurrency_test.sql`). Session A runs `SELECT ... FOR UPDATE` on a slot row and keeps its transaction open. Session B runs the same statement and is blocked until A finishes. The same logic was also tried through the app, where two users press **Confirm & Pay** for one slot (for example two requests for slot_id 8, Turf A at GreenPark Football Arena, Kollam): the first request got HTTP 200 and the second got HTTP 409. Table 4.1 compares what happens with and without the lock.

Step	Without concurrency control	Slotit (row-level locking)
Both sessions read the slot	Both see 'available'	Session B waits on the row lock
Session A writes	Sets 'booked' and inserts a booking	Sets 'booked', inserts the booking and commits
Session B continues	Also sets 'booked' and inserts a second booking	Reads the row again, sees 'booked' and rolls back
Bookings for the slot	2 (double booking)	1
Wallet of user B	Charged for an invalid booking	Not changed
Reply to user B	Success (wrong)	'Slot is no longer available' (HTTP 409)

Table 4.1: Behaviour with and without row locking (two-session test)

The result matches what we expected: the lock removes the gap between reading and writing, so only one booking is created and the second user gets a clear message.

4.3 Wallet, Cancellation and Refund

When a user books a slot, the wallet balance goes down by the final price, the slot becomes 'booked' and a booking row appears. When the booking is cancelled, the booking becomes 'refunded', the slot becomes 'available' and the same amount comes back to the wallet. When the wallet did not have enough balance, the booking was rejected and no table was changed, which shows atomicity. Cancelling a past booking or an already refunded booking was also rejected.

4.4 Dynamic Pricing and Waitlist Notification

On the venue page, evening slots (17:00 to 21:00) were shown with a Peak badge and the higher price, while daytime slots showed the Off-Peak Deal badge with the base price. The price shown on the screen matched the price calculated by the SQL query and charged at checkout.

To test the trigger, a user was added to the waitlist of a booked slot and the booking was then cancelled. As soon as the slot changed to 'available', `trg_slot_drop_notify` inserted a row in the notifications table for that waitlisted user. Similarly, `trg_update_streak` increased the streak_count of the player after each new booking.

4.5 Geospatial Search

`VenueDAO.findNearby` calculates the distance with the Haversine expression and filters with `HAVING distance_km <= radius`. The seed data has venues in Kollam, Kochi, Trivandrum and Calicut, and a composite index on (latitude, longitude) is created in schema.sql. Table 4.2 shows the radius values used in the app.

Search radius	Filter applied	Sort order	Supporting index
5 km	distance_km <= 5	Nearest first	idx_venue_location
15 km	distance_km <= 15	Nearest first	idx_venue_location
25 km	distance_km <= 25	Nearest first	idx_venue_location
50 km	distance_km <= 50	Nearest first	idx_venue_location

Table 4.2: Haversine radius search behaviour

Because of the `LEAST/GREATEST` clamp, no error came from rounding in `acos()`. If GPS is not allowed, the user can simply choose a city from the location screen.

4.6 Tournament and Squad Features

When a team was registered, the tournament row was locked first, so the team limit (`max_teams`) was never crossed, and a team name that already existed (in any letter case) was rejected with a 409 reply. Bracket generation refused to run when fewer than two teams were registered or when fixtures already existed. For squads, a player could not join the same squad twice, and the squad status changed to 'full' when the last place was taken.

4.7 Offline Behaviour of the App

The service worker pre-caches the application shell (`slotit-app-v5`). When the server was stopped or the Wi-Fi was disconnected, requests were cancelled after 900 ms and the app switched to the built-in venue catalogue for two minutes, so the screens stayed usable instead of showing a blank loading page. On Android Chrome the site could be added to the home screen and opened like an app.

4.8 Data Loaded in the Database

The seed script fills the database so that every screen has something to show. Table 4.3 gives a summary.

Table	Rows	Details
users	5	2 players, 2 venue owners and 1 admin
venues	17	Sample venues: Kollam 5, Kochi 4, Trivandrum 4, Calicut 4
courts	34	2 courts per venue, rates from Rs 300 to Rs 1,000 per hour, peak multipliers from 1.25 to 1.50
time_slots	2,618	11 hourly slots (06:00 to 21:00) for 34 courts over 7 days; evening slots flagged as peak
tournaments and teams	3 and 3	Kollam Super 5s Cup (Football), SmashPoint Open (Badminton), KCL Box Cricket League (Cricket)
squad_requests	3	Open squads for football, badminton and cricket

Table 4.3: Sample data in slotit_db

4.9 Challenges and Solutions

Problem	Reason	Solution
Deadlock in bookings	Two transactions locked the slot and the wallet in opposite order	Fixed a lock order: time_slots row first, then users row
Wrong distances	FLOAT lost precision for coordinates, and acos() could get a value just outside -1 to 1	Used DECIMAL(10,8) and DECIMAL(11,8) for latitude and longitude, and clamped with LEAST and GREATEST
CORS errors on phone	The browser blocked calls from the phone to Tomcat	Added CORS headers and doOptions handlers in all 9 servlets
No JSON library	We wanted to avoid extra JAR files	Wrote a small JSON parser and escaper in JsonUtil.java; models use locale-safe toJson()
Password in code	Database password was hard coded	Moved credentials to environment variables set by setenv.bat
Page hangs on weak Wi-Fi	The browser waits for a response for a long time	900 ms timeout, 30-second cache and 2-minute circuit breaker
Phone could not reach the server	Windows Firewall blocked port 8085	Launcher script adds a firewall rule for port 8085
PWA install blocked on Chrome (Android)	Chrome wants HTTPS or localhost and PNG icons for installation	Added 192x192 and 512x512 PNG icons, registered sw.js, and used the insecure-origin flag for the lab address

Table 4.4: Problems faced and how they were solved

4.10 Limitations and Future Scope

- MongoDB is not yet connected to the Java code. `UserStatsDAO` returns mock data and the driver calls are commented out.
- Load and timing tests were not done, so only the correctness of the locking was verified.

- A few derived columns (badge, total_bookings, streak_count, is_peak, current_count, player_count) are stored on purpose, so the schema is 3NF with some controlled denormalization.
- The wallet is an in-app balance; a real payment gateway can be added later.
- The app runs on plain HTTP in a local network. HTTPS and a cloud server would be needed for public use.
- Only round-1 knockout fixtures are generated. Later rounds, league tables and live score updates can be added.
- The admin page is only an overview and can be extended with real approval controls.

4.11 Conclusion

Slotit achieved the objectives of the project. The 3NF schema keeps the data clean, row-level locking removes the double-booking problem, transactions keep the wallet and slots consistent, and the triggers, Haversine search, HotSlot pricing, tournament module, squad finder and the offline-ready PWA make it a complete working system. The main work left is to connect MongoDB fully and to test the system under heavy load.

References

1. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill Education, 2019.
2. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.
3. R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed., McGraw-Hill, 2003.
4. C. J. Date, An Introduction to Database Systems, 8th ed., Addison-Wesley, 2004.
5. Oracle Corporation, MySQL 8.0 Reference Manual: InnoDB Locking and Transaction Model. https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html
6. MongoDB Inc., The MongoDB Architecture Guide: Polyglot Persistence and Modern Data Platforms.
7. World Wide Web Consortium (W3C), Service Workers. https://www.w3.org/TR/service-workers/
8. Apache Software Foundation, Apache Tomcat 9 Documentation. https://tomcat.apache.org/tomcat-9.0-doc/
9. OpenWeatherMap, 5 day / 3 hour Forecast API Documentation. https://openweathermap.org/forecast5
10. Oracle Corporation / Eclipse Foundation, Java Servlet 4.0 (Java EE 8) and JDBC API Specifications.
11. Mozilla Developer Network, Progressive Web Apps: Web App Manifest and Service Workers. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

Appendix

A. Key Equations

- **Haversine distance:** d = 2R x arcsin(sqrt(sin²(Δφ/2) + cos φ1 x cos φ2 x sin²(Δλ/2))), where R = 6371 km and φ, λ are latitude and longitude in radians. (The SQL uses the spherical law of cosines form, R x acos(...), which gives the same distance.)
- **Knockout bracket:** For N teams, number of rounds = ceil(log2 N) and total matches = N - 1.
- **Peak price:** final_price = hourly_rate x (peak_multiplier if is_peak, else 1).
- **Loyalty badge:** silver at 5 bookings, gold at 10, platinum at 25.

B. REST API Summary

Servlet	Method and path	Input	Result
LoginServlet	POST /api/auth/login	email, password	User details or 401
RegisterServlet	POST /api/auth/register	name, email, phone, password, role	201 Created (role limited to player or owner)
VenueServlet	GET /api/venues	id, sport, or lat, lng, radius	Venue(s), with distance when lat and lng are given
SlotServlet	GET /api/courts, GET /api/slots	venue_id, date	Courts, or slots with finalPrice
BookingServlet	POST and DELETE /api/bookings	slot_id, user_id / booking_id, user_id	Locked booking (200 or 409) / cancel with refund
MyBookingsServlet	GET /api/bookings/my	user_id, type=upcoming or past	User's bookings
OwnerDashboardServlet	GET /api/owner/dashboard	owner_id	KPIs, recent bookings, 7-day revenue
TournamentServlet	GET /api/tournaments, GET /api/tournaments/teams, POST /api/tournaments/register	tournament_id, team_name, captain_id	Tournaments, teams, or new team (409 on conflict)
SquadServlet	GET /api/squads, POST /api/squads/create, POST /api/squads/join	user_id, slot_id, sport, players_needed, message / request_id	Open squads, new request or joined squad

Table A.1: REST API endpoints

C. Software Tools Used

Component	Tool	Version / detail	Use
Relational database	MySQL Community Server	8.0	ACID transactions, 3NF schema, row locks, triggers
NoSQL database	MongoDB Community	Setup script	Player stats, commentary, photos
Application server	Apache Tomcat	9.0.122 (port 8085)	Runs the servlets
Language	Java	Compiled with JDK 17 for Java 11	Servlets, DAOs, JDBC
Build tool	Apache Maven	3.9.6	Builds Slotit.war
Libraries	javax.servlet-api and mysql-connector-j	4.0.1 and 8.0.33	Servlet API and JDBC driver

Component	Tool	Version / detail	Use
Frontend	HTML5, CSS3, JavaScript (ES6)	PWA with service worker	User interface
Weather	OpenWeatherMap API	5-day / 3-hour forecast	Weather advisory
Operating system	Windows 10 / 11	64-bit	Host machine for the server

Table A.2: Software tools and versions

D. Viva Queries (useful_queries.sql)

No.	What the query shows	SQL ideas used
1	Bookings with user, slot, court and venue	Multi-table JOIN
2	Revenue of each venue	GROUP BY and SUM
3	Loyal users with more than one booking	HAVING
4	Venues priced above the average	Subquery
5	Peak and off-peak utilization	CASE expression
6	Waitlist demand ranking	Aggregation and ORDER BY
7	Tournament standings and wins	JOIN and aggregation
8	Active upcoming bookings	View: active_upcoming_bookings
9	Open seats in squads	Arithmetic on columns
10	Customer lifetime value ranking	SUM and ranking

Table A.3: The ten analytical queries

E. Project File Structure

Path	Contents
Slotit/pom.xml	Maven build file (Java 11, WAR packaging)
Slotit/database/	schema.sql, triggers.sql, useful_queries.sql, mongodb_setup.js, concurrency_test.sql
Slotit/src/main/java/com/slotit/models/	5 model classes
Slotit/src/main/java/com/slotit/dao/	7 DAO classes
Slotit/src/main/java/com/slotit/servlets/	9 servlets
Slotit/src/main/java/com/slotit/util/	DBConnection, PasswordUtil, JsonUtil
Slotit/src/main/webapp/	14 pages, css/style.css, js/app.js, js/hotslot.js, js/weather.js, manifest.json, sw.js, icons
Start_Slotit_Server.bat	One-click launcher for MySQL, firewall rule and Tomcat

Table A.4: Files of the project

F. Database Table Specifications (Data Dictionary)

Column	Data type and constraints	Description
user_id	INT PRIMARY KEY AUTO_INCREMENT	Unique ID of each account
name	VARCHAR(100) NOT NULL	Full name of the player, owner or admin
email	VARCHAR(150) UNIQUE NOT NULL	Login email; indexed by idx_user_email
phone	VARCHAR(15) NOT NULL	10-digit mobile number

Column	Data type and constraints	Description
password_hash	VARCHAR(255) NOT NULL	BCrypt ($2a$10$) salted hash made by PasswordUtil.java
role	ENUM('player','owner','admin')	Decides access to player screens or owner dashboard
wallet_balance	DECIMAL(10,2) DEFAULT 0.00	In-app wallet used for payments and refunds
streak_count	INT DEFAULT 0	Increased by 1 by trg_update_streak after every booking
total_bookings	INT DEFAULT 0	Lifetime bookings, increased on every confirmed booking
badge	ENUM('none','silver','gold','platinum')	Loyalty tier shown on the profile screen

Table A.5: Specification of the users table

Table.column	Data type and constraints	Description
venues.venue_id	INT PRIMARY KEY AUTO_INCREMENT	Unique ID of a sports complex (17 venues)
venues.owner_id	INT FK -> users(user_id)	Owner who manages the venue dashboard
venues.name, city	VARCHAR(150), VARCHAR(50)	Venue name and city (Kollam, Kochi, Trivandrum, Calicut)
venues.latitude, longitude	DECIMAL(10,8), DECIMAL(11,8)	GPS position; indexed by idx_venue_location
venues.sport_type	ENUM('football','cricket','badminton','tennis','basketball')	Main sport, used by the filter chips
venues.is_outdoor	BOOLEAN DEFAULT TRUE	Used by weather.js to show rain warnings only for outdoor turfs
courts.court_id	INT PRIMARY KEY AUTO_INCREMENT	Unique ID of a court or turf (34 courts)
courts.venue_id	INT FK -> venues(venue_id) ON DELETE CASCADE	Venue that owns the court
courts.hourly_rate	DECIMAL(8,2) NOT NULL	Base price per hour (Rs 300 to Rs 1,000)
courts.peak_multiplier	DECIMAL(3,2) DEFAULT 1.50	HotSlot multiplier for peak evening hours

Table A.6: Specification of the venues and courts tables

Table.column	Data type and constraints	Description
time_slots.slot_id	INT PRIMARY KEY AUTO_INCREMENT	Unique ID of a one-hour slot (2,618 seeded slots)
time_slots.court_id	INT FK -> courts(court_id)	Court the slot belongs to
time_slots.slot_date	DATE NOT NULL	Date of the slot; indexed by idx_slot_date_status
time_slots.start_time, end_time	TIME NOT NULL	For example 18:00:00 to 19:00:00; UNIQUE(court_id, slot_date, start_time)
time_slots.status	ENUM('available','booked','maintenance')	Reservation state; locked with FOR UPDATE at checkout
time_slots.is_peak	BOOLEAN DEFAULT FALSE	TRUE for 17:00 to 21:00; triggers HotSlot pricing

Table.column	Data type and constraints	Description
bookings.booking_id	INT PRIMARY KEY AUTO_INCREMENT	Booking receipt number
bookings.user_id, slot_id	INT FKs -> users, time_slots	Player and the slot that was reserved
bookings.amount_paid	DECIMAL(8,2) NOT NULL	Price deducted from the wallet (rate x multiplier if peak)
bookings.payment_status	ENUM('pending','completed','refunded')	'completed' means active, 'refunded' means cancelled

Table A.7: Specification of the time_slots and bookings tables

Table	Keys	Purpose and key columns
tournaments	PK tournament_id; FK venue_id, created_by	name, sport, start_date, end_date, format (knockout or league), max_teams (default 8), status (upcoming, ongoing, completed)
teams	PK team_id; FK tournament_id, captain_id	Registered teams with name and player_count; locked during registration to stop duplicate names and extra teams
matches	PK match_id; FK tournament_id, team1_id, team2_id, court_id, winner_id	Knockout fixtures made by generateKnockoutFixtures(), scores (score_team1, score_team2) and match_status
squad_requests	PK request_id; FK user_id, slot_id	Open teammate posts: sport, players_needed, current_count, message, status (open, full, cancelled)
squad_members	PK id; FK request_id, user_id	Players who joined a squad; UNIQUE(request_id, user_id) stops joining twice
waitlist	PK waitlist_id; FK user_id, slot_id	Users waiting for a booked slot to open
notifications	PK notification_id; FK user_id	Alerts inserted by trg_slot_drop_notify

Table A.8: Specification of the remaining seven tables

G. Live Verification Queries (to show database changes during the viva)

```
-- Latest bookings and wallet balances
SELECT b.booking_id, u.name, u.wallet_balance, u.streak_count,
       ts.slot_date, ts.start_time, ts.status, b.amount_paid, b.payment_status
FROM slotit_db.bookings b
JOIN slotit_db.users u ON b.user_id = u.user_id
JOIN slotit_db.time_slots ts ON b.slot_id = ts.slot_id
ORDER BY b.booking_id DESC LIMIT 5;

-- Newly registered users
SELECT user_id, name, email, role, wallet_balance, badge
FROM slotit_db.users ORDER BY user_id DESC LIMIT 5;

-- Squad matchmaking updates
SELECT request_id, sport, players_needed, current_count, status, message
FROM slotit_db.squad_requests ORDER BY request_id DESC LIMIT 5;
```